Automated Data-Integrity Checks

How the three verification goals are checked **programmatically** instead of by eye, so a human reviews only the flagged exceptions. Korean mirror: `DATA_INTEGRITY_CHECKS_KR.md`. Source of truth: `app/Backend/integrity_checks.py`.

The three goals map 1:1 to three checkers:

Goal	Checker	Function
1. All user activities recorded (Admin page)	**C3** activity cross-check (+ C1 behavior invariants)	`check_activity_crosscheck`
2. All data correctly stored in the DB	**C1** DB integrity invariants	`check_db_integrity`
3. All data correctly stored in REDCap	**C2** DB↔REDCap reconciliation	`check_redcap_reconciliation`

Every check returns a `CheckResult(name, status, count, total, detail, examples[])` where `status ∈ {pass, warn, fail}`. `run_all_checks` aggregates: any `fail` → overall `fail`; else any `warn` → `warn`; else `pass`. All checks are **read-only** (no DB or REDCap writes).

Goal 1 — user activities recorded on the Admin page

The admin pages just display the three behavior tables, so "recorded on the admin page" = "the event landed in `patient_report_page_behavior` / `patient_followup_survey_page_behavior` / `doctor_behavior`". Tracking is **best-effort** (fire-and-forget, no retry; `session_end` fires on `beforeunload`), so completeness can only be *measured*, not proven from the DB alone. Two mechanisms:

C3 — cross-check against the canonical record (the key one)

`check_activity_crosscheck` treats the survey **answer** (`patient_survey_submission_log`) as ground truth and asks whether a corresponding **behavior event** exists:

```
for each survey submission (file, speaker):
    is there any behavior event of type
      survey_answer | survey_complete | domain_submitted
    for the same (file, speaker)?
```

- Implemented as set difference: pull the `(file, speaker)` set that produced those event types, then list submissions whose `(file, speaker)` is **not** in it → `activity_survey_trail` (status `warn`, with examples). A submission with no matching behavior event is a likely **silent drop**.

C1 — behavior-stream completeness (soft)

Inside `check_db_integrity`, per `session_id`: - `behavior_session_missing_page_view` — sessions with no `page_view` start marker (`warn`). - `behavior_session_missing_session_end` — sessions with no `session_end` (`warn`; unload-time loss is expected). - `behavior_close_without_open` — more `*_close` than `*_open` (topic/evidence/summary) in a session (`warn`; usually a dropped `open` event). These are reported as **anomaly rates** (count / total_sessions), not hard failures.

Goal 2 — data correctly stored in the DB

`check_db_integrity` asserts invariants over `patient_survey_submission_log` (+ recordings). Each is a count of violating rows that **should be 0**:

Check	Logic	Status if violated
`survey_orphan_rows`	survey LEFT JOIN `patient_summary` on (file, speaker) → parent missing	`fail`
`survey_empty_answers`	`answers` is `{}` / null	`fail`
`survey_bad_type`	`survey_type` ∉ {dcs, sdm, satisfaction, risk_perception, risk_perception_2}	`fail`
`survey_synced_missing_record_id`	`redcap_synced = true` but `redcap_record_id` is null	`fail`
`survey_sid_unresolvable`	`unhash_patient_sid(speaker)` returns null (attribution broken)	`warn`
`recording_bad_area`	`session_recording.area` ∉ the 7 allowed values	`fail`

Rationale: the DB is transactional (a row existing = it was stored), but *correctness* — referential integrity (orphans), completeness (empty answers), and attribution (SID resolvable) — needs explicit assertion. Each violation ships up to 10 example rows.

Goal 3 — data correctly stored in REDCap

`check_redcap_reconciliation` compares each synced submission against the actual REDCap record (read-only export). It **reuses the exact production mapping code**, so it checks what REDCap really holds, not what we assumed we sent:

```
for each submission where redcap_synced = true:
  1. record_id = redcap_record_id  (= SID from unhash_patient_sid(speaker))
  2. EXPECTED {redcap_field: value} from the DB answers:
       follow-up (dcs/sdm/risk_perception/satisfaction):
           FRONTEND_TO_REDCAP_MAPPING[survey_type] + transform_value(...)
       first-visit risk_perception_2:
           _fv_answer_to_redcap(question_id, field, value)
  3. ACTUAL = export that record from REDCap (one read-only httpx POST, batched by record_id)
  4. compare EXPECTED vs ACTUAL field by field
```

- `redcap_missing_record` — the DB says `synced` but REDCap has no such record (the 13475-style miss) → `fail`.
- `redcap_field_mismatch` — a REDCap field value differs from the DB-derived expected value → `fail`.
- Unconfigured REDCap → `warn` (skipped).

Because EXPECTED is built with the *same* `FRONTEND_TO_REDCAP_MAPPING` / `transform_value` / `_fv_answer_to_redcap` used at write time (`routes_surveys.py`, `routes_patient.py`), value coding (SDM yes/no→1/0, DCS 0-based→1-based, sliders→categories, checkbox `field___code`) is applied identically on both sides.

How to run it (three surfaces)

Surface	Command / URL	Behavior
CLI	`python scripts/verify_integrity.py` (`--json`, `--check c1|c2|c3`, `--skip-redcap`)	prints a report; **exit 1** if any check fails
HTTP	`GET /api/admin/integrity` (admin-guarded)	JSON report; **HTTP 503** if overall = fail, else 200
Admin UI	`/admin/tracking/data-integrity`	one row per check (green/amber/red) + expandable **exception examples only**

A human reviews only the amber/red rows and their example lists — not every submission.

Key files

- `app/Backend/integrity_checks.py` — the three checkers + `run_all_checks`.
- `app/Backend/scripts/verify_integrity.py` — CLI.
- `app/Backend/routes_admin_integrity.py` — `GET /api/admin/integrity`.
- `app/Webapp/src/components/AdminDataIntegrity.tsx` — admin UI panel.
- Reused: `deid.unhash_patient_sid`, `routes_surveys.FRONTEND_TO_REDCAP_MAPPING` / `transform_value`, `routes_patient._fv_answer_to_redcap`.